

Mirea CRM

Микросервисная система для малого бизнеса сферы услуг

Описание архитектуры

Микросервисов	8
Языки	Python, Go
Синхронные вызовы	9 методов gRPC
Доменные события	10
Брокеры сообщений	RabbitMQ, NATS
Автоматических тестов	287

Содержание

1 Назначение системы	3
2 Декомпозиция на сервисы	3
3 Топология системы	4
4 Транспортные уровни	5
5 Доменные события	6
6 Организация данных	7
7 Сквозные решения	7
7.1 Контракты прежде реализации	8
7.2 Непрерывность трассировки	8
7.3 Идемпотентность потребителей	8
7.4 Единая трактовка ошибок	8
7.5 Разделение живости и готовности	9
7.6 Разграничение доступа	9
7.7 Наблюдаемость	10
7.8 Непрерывная интеграция и доставка	11
8 Сквозные сценарии	12
8.1 Запись клиента	12
8.2 Завершение визита	13
9 Описание сервисов	14
9.1 core-service	14
9.2 catalog-service	14
9.3 booking-service	15
9.4 inventory-service	16
9.5 client-service	17
9.6 billing-service	18
9.7 notification-service	19
9.8 analytics-service	20
9.9 gateway	21
10 Незавершённые работы	22

1 Назначение системы

Сеть небольших предприятий сферы услуг ведёт филиалы и специалистов. Клиенты записываются на услуги, каждая занимает известное время и расходует материалы. После визита выставляется счёт, специалисту начисляется комиссия, клиенту — баллы лояльности. Владелец сети смотрит выручку, загрузку специалистов и расход материалов.

Модель нейтральна к нише. Она одинаково ложится на барбершоп, тату-студию, автосервис, частную клинику и мастерскую по ремонту — везде, где клиент записывается к специалисту на услугу определённой длительности. Термин «специалист» выбран именно поэтому: в барбершопе его назовут мастером, в клинике врачом, в автосервисе механиком, а в модели это одна роль.

Границы системы

То, чего система намеренно не делает, определяет её не меньше, чем реализованные функции:

- эквайринга нет — оплата счёта отмечается вручную;
- реальной отправки сообщений нет — адаптеры каналов пишут в журнал;
- закупок и поставщиков нет — склад знает остатки, но не знает их происхождения;
- интерфейса пользователя нет — система предоставляет программный интерфейс.

Ограничения приняты для того, чтобы объём работы остался учебным, а архитектурные решения — производственными.

2 Декомпозиция на сервисы

Разделение выполнено по ограниченным контекстам предметной области, а не по техническим слоям. В системе нет сервиса «база данных» или сервиса «интерфейс» — есть сервис, отвечающий за записи, и сервис, отвечающий за склад. Граница проходит там, где меняется язык предметной области.

У каждого контекста один владелец данных. Если запись создаёт сервис booking, то и историей визитов владеет он — карточка клиента получает её запросом, а не хранит копию. Если контакты клиента находятся в client, то notification запрашивает их в момент отправки. Правило формулируется одной фразой: **дублировать чужие данные нельзя, можно только запрашивать.**

Контекст	Сервис	Язык	Область ответственности
Организация	core	Python	компании, филиалы, сотрудники, графики работы
Каталог	catalog	Python	услуги, цены по филиалам, нормативы расхода
Записи	booking	Go	запись к специалисту, расписание, свободные слоты
Склад	inventory	Go	остатки материалов, списание, минимальные пороги
Клиенты	client	Python	карточка клиента, контакты, лояльность
Расчёты	billing	Python	счета, оплаты, комиссия специалиста
Уведомления	notification	Go	напоминания клиентам, алерты администраторам
Аналитика	analytics	Python	выручка, загрузка, конверсия, расход материалов

Таблица 1 — Ограниченные контексты и соответствующие им сервисы

Выбор языка реализации

Go применён там, где он уместен по существу задачи. Захват слота — конкурентная операция: два администратора могут инициировать запись на одно время одновременно. Списание материалов — конкурентное изменение счётчика. Рассылщик уведомлений должен быть лёгким и удерживать два постоянных соединения с брокерами. Остальные сервисы — справочники и отчёты, где выигрывает скорость разработки на Python.

Следствие видно в размере образов: сервисы на Go, собранные на distroless, занимают 28–34 МБ против 332 МБ у Python. Холодный старт — десятки миллисекунд против нескольких секунд, поэтому именно сервис на Go назначен целью для бессерверного развёртывания.

3 Топология системы

Система состоит из двух ярусов и двух шин. Верхний ярус — сервисы, к которым обращаются синхронно; нижний — потребители событий. Наружу смотрит только шлюз, публичных портов у сервисов нет.

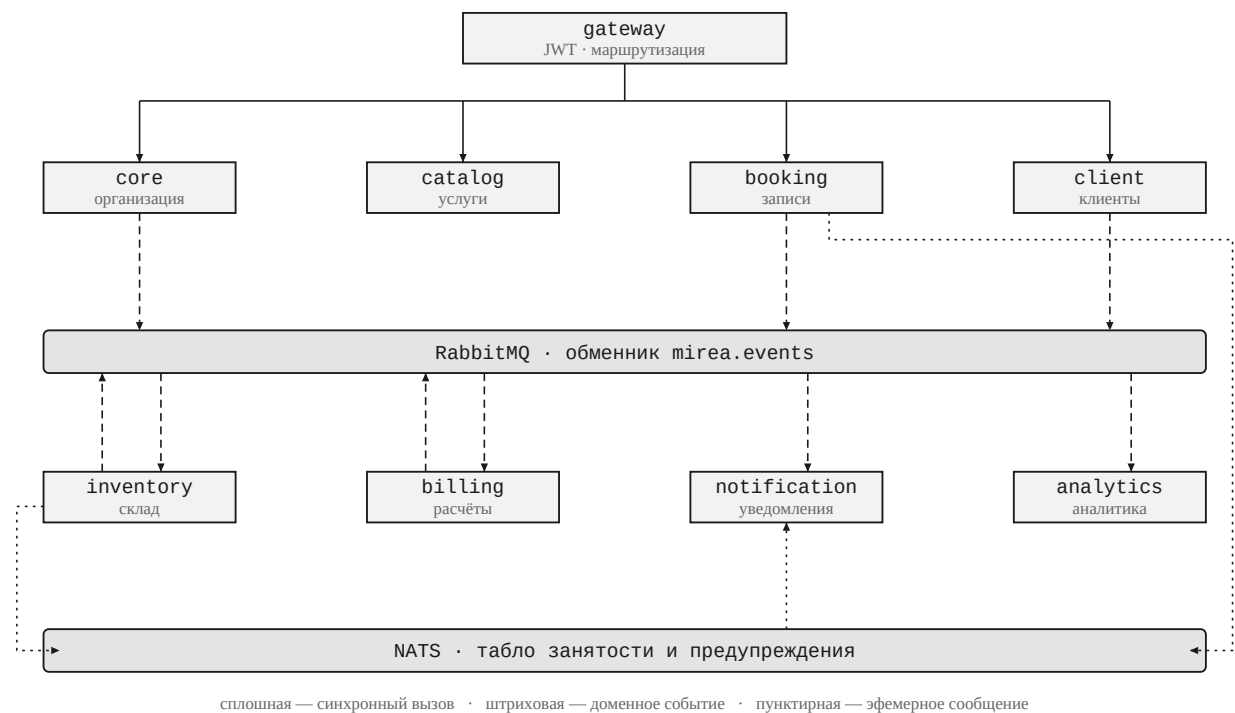
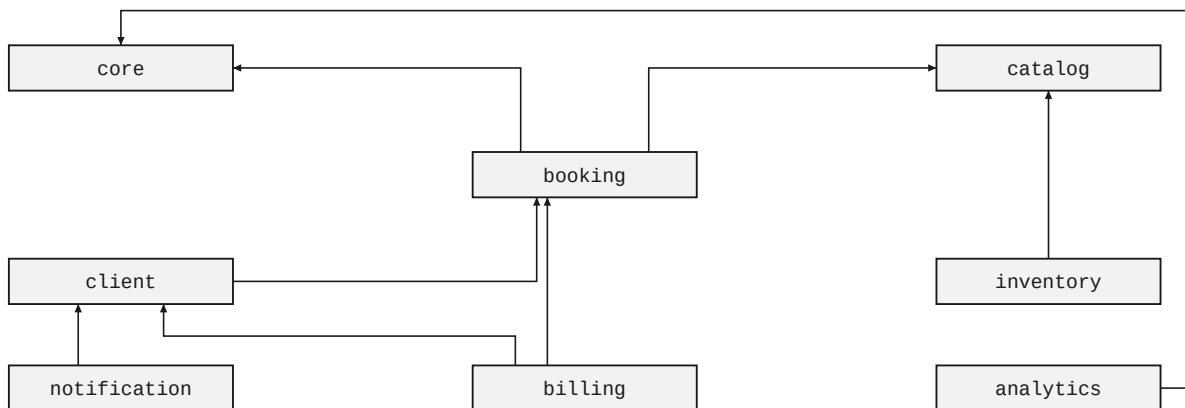


Рисунок 1 — Ярусы системы и шины сообщений

Синхронные обращения между сервисами образуют отдельный граф. Он намеренно разрежен: чем меньше синхронных зависимостей, тем меньше вероятность, что отказ одного сервиса остановит другой.



граф ацикличен: ни один сервис не вызывает того, кто вызывает его

Рисунок 2 — Граф синхронных вызовов: стрелка от вызывающего к вызываемому

Вызов	Метод	Назначение
booking → core	GetEmployeeSchedule	график специалиста
booking → core	GetBranch	проверка филиала
booking → catalog	GetService	длительность и цена услуги
inventory → catalog	GetConsumptionNorms	норматив расхода материалов
billing → booking	GetAppointment	детали визита
billing → client	AddLoyaltyPoints	начисление баллов
client → booking	ListClientAppointments	история записей
notification → client	GetClientContacts	контакты и канал связи
analytics → core	GetBranch, ListEmployees	справочник для отчётов

Таблица 2 — Синхронные вызовы между сервисами

4 Транспортные уровни

В системе четыре канала передачи данных. Каждый отвечает за свой класс задач, и правило выбора формулируется одной фразой — это существеннее самих технологий.

Канал	Область применения	Правило выбора
REST	запросы извне, через шлюз	идёт мимо шлюза — значит не REST
gRPC	синхронные вызовы между сервисами	нужен ответ прямо сейчас
RabbitMQ	доменные события, меняющие состояние	потеря сообщения означает потерю данных
NATS	эфемерные широковещательные обновления	сообщение устаревает за секунды

Таблица 3 — Транспортные уровни и правила выбора

Внешний интерфейс

Всё, что приходит от человека, проходит через шлюз по протоколу HTTP с телом в формате JSON. Единственный HTTP-переход внутри периметра — от шлюза к сервису.

Синхронное взаимодействие

Обращения к соседним сервисам выполняются по gRPC: девять методов, все внутренние. Контракты описаны в файлах формата Protocol Buffers и генерируются в оба языка, поэтому рассогласование обнаруживается при сборке, а не во время работы.

Гарантированная доставка событий

Всё, что меняет состояние системы и не может быть потеряно, публикуется в обменник типа topic mirea.events. Очереди объявлены как durable, издатель работает с подтверждениями, потребитель — с ручным подтверждением обработки. Неразобранные сообщения направляются в отдельную очередь недоставленных.

Эфемерные сообщения

Широковещательные обновления, потеря которых безвредна, передаются через NATS: изменение занятости на табло у стойки регистрации и всплывающие предупреждения администратору. Гарантий доставки нет, подписчик синхронизирует состояние при переподключении.

О наличии двух брокеров. RabbitMQ обеспечивает гарантии доставки ценой очередей и подтверждений; NATS обеспечивает скорость ценой отсутствия гарантий. Наглядно различие видно на предупреждении о низком остатке материала: один и тот же факт передаётся обоими путями — через NATS мгновенно на экран, через RabbitMQ надёжно для отчётности и уведомления по электронной почте. В промышленной системе такого масштаба достаточно одного брокера; второй введён для демонстрации обоих классов доставки, и слой NATS изолирован так, что удаляется без изменения доменной логики.

5 Доменные события

В системе десять доменных событий. Имя каждого — глагол в прошедшем времени: событие уже произошло, отменить его нельзя, можно лишь выпустить другое, компенсирующее его последствия.

Событие	Публикует	Потребляют
branch.opened	core	analytics
employee.hired	core	analytics
client.registered	client	analytics
appointment.created	booking	notification, analytics
appointment.cancelled	booking	notification, analytics
appointment.completed	booking	inventory, billing, analytics
consumables.written_off	inventory	analytics
stock.low	inventory	notification, analytics
invoice.issued	billing	notification, analytics
invoice.paid	billing	analytics

Таблица 4 — Каталог доменных событий

Ключевым является событие appointment.completed. Одно событие расходится на трёх потребителей: склад списывает материалы, расчётный сервис выставляет счёт, аналитика учитывает выручку. Ни один из них не знает о существовании остальных, и добавление четвёртого

потребителя не требует изменений в сервисе booking.

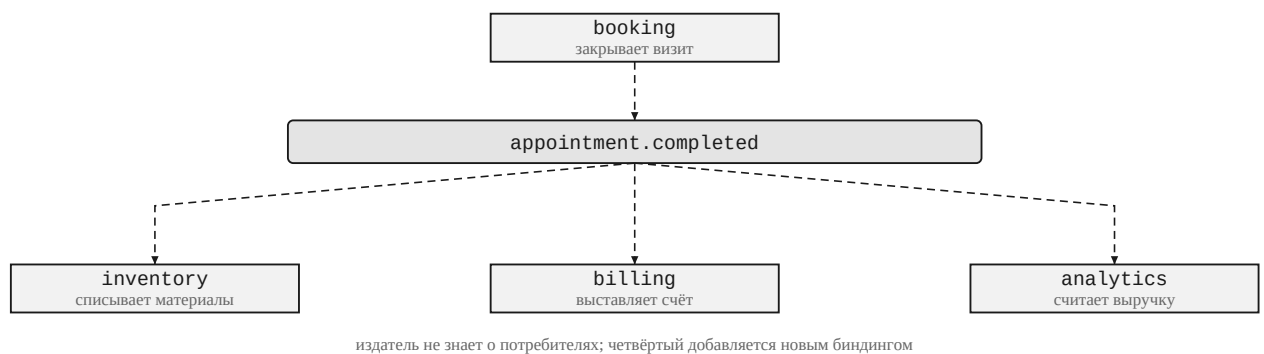


Рисунок 3 — Расхождение одного события на независимых потребителей

Состав сообщения

Событие несёт идентификаторы, а не полный снимок данных. Расчётный сервис получает идентификатор записи и самостоятельно запрашивает цену и специалиста. Преимущество: полезная нагрузка не разрастается и не требует версионирования при каждом изменении карточки визита. Недостаток: потребитель зависит от доступности издателя в момент обработки. Размен принят осознанно.

Формат представления

Схемы событий описаны в Protocol Buffers, однако в очередь помещается не двоичное представление, а JSON, полученный по правилам protobuf-JSON. Плата — размер сообщения: 289 байт против 116. Приобретение — читаемость: когда сервис на Go не может разобрать событие от сервиса на Python, в панели управления брокером виден текст, а не шестнадцатеричный дамп. При восьми сервисах на двух языках это окупается.

6 Организация данных

Каждому сервису соответствует отдельная база данных с собственным пользователем. Изоляция не декларативная: попытка пользователя одного сервиса открыть чужую базу отклоняется сервером с ошибкой доступа.

Физически используется один экземпляр PostgreSQL. Восемь отдельных экземпляров избыточны для учебного стенда, при этом требование «база на сервис» соблюдено, и в контекстной карте базы изображаются отдельно.

Схема во всех сервисах поднимается миграциями, а не автоматическим созданием таблиц: в Python применяется Alembic, в Go — goose со встроенными в исполняемый файл SQL-сценариями. Миграции накатываются при запуске контейнера, поэтому отдельный шаг развёртывания не требуется.

Особенность Alembic. Автоматический генератор миграций помещает в процедуру отката только удаление таблиц, тогда как перечислимый тип остаётся в базе. Повторный накат завершается ошибкой существующего объекта. Проявляется не сразу: первый прогон на чистой базе проходит успешно. Во всех сервисах, использующих перечислимые типы, удаление дописано вручную.

7 Сквозные решения

Восемь механизмов реализованы одинаково во всех сервисах. Именно они превращают набор программ в систему.

7.1 Контракты прежде реализации

Каталог с описаниями Protocol Buffers — единственный источник сведений о границах сервисов. Из него генерируются и классы Python, и структуры Go, поэтому несовпадение приводит к ошибке сборки. Практическая польза оказалась шире ожидаемой: сервис записей разрабатывался и тестировался против подставного каталога услуг, когда настоящий ещё не существовал.

Номера полей в описании и составляют контракт: по каналу связи передаётся номер, а не имя. Поле допустимо переименовать, номер — нет.

7.2 Непрерывность трассировки

Заголовок `tracereparent` принимается из входящего HTTP-запроса, передаётся в метаданных gRPC, проставляется в конверт события и восстанавливается потребителем на другом конце очереди. Один идентификатор проходит через синхронные вызовы, оба брокера и оба языка.

Без этого события в системе трассировки выглядели бы несвязанными корнями, и установить, какой запрос породил списание материалов, было бы нечем.

7.3 Идемпотентность потребителей

RabbitMQ гарантирует доставку не менее одного раза, поэтому повторная доставка неизбежна. У каждого потребителя предусмотрена таблица обработанных событий: обработка начинается со вставки идентификатора.

Отметка записывается той же транзакцией, что и результат обработки. Это не деталь реализации, а суть механизма. При записи отметки отдельной транзакцией неудачная обработка оставит событие помеченным, и повторная доставка молча его пропустит. Дефект был обнаружен именно так: списание не прошло из-за нехватки материала, а событие уже считалось обработанным.

7.4 Единая трактовка ошибок

Доменные ошибки объявлены один раз: «не найдено», «конфликт состояния», «недопустимый аргумент», «сосед недоступен». Каждый транспорт переводит их в собственные коды по таблице соответствия. В обработчиках запросов разбор ошибок отсутствует.

Доменная ошибка	HTTP	gRPC
Не найдено	404	NOT_FOUND
Конфликт состояния	409	ABORTED
Недопустимый аргумент	422	INVALID_ARGUMENT
Сосед недоступен	503	UNAVAILABLE
Нарушение ограничения БД	409	ABORTED

Таблица 5 — Соответствие доменных ошибок кодам транспортов

Отдельного внимания заслуживает «сосед недоступен». Если недоступен `catalog`, это не внутренняя ошибка `booking`, и код 500 неверен: клиенту имеет смысл повторить запрос, а системе мониторинга — не относить отказ на наш счёт.

7.5 Разделение живости и готовности

Проба `/healthz` отвечает «процесс не завис» и зависимости не проверяет. Проба `/readyz` проверяет базу данных и брокеры и возвращает код 503, если что-либо недоступно. При объединении проб оркестратор либо считает сервис живым при недоступной базе, либо перезапускает исправный сервис из-за отказа соседа.

7.6 Разграничение доступа

Проверка прав выполняется однократно, на шлюзе. Методические указания допускают и проверку в каждом сервисе, обращающемся к системе управления доступом, однако при восьми сервисах это означало бы восемь копий одинакового кода и восемь мест, где его можно реализовать по-разному.

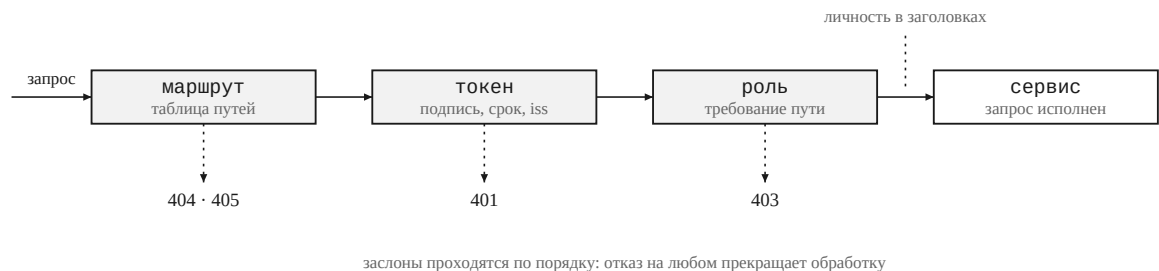


Рисунок 4 — Прохождение запроса через шлюз и коды отказа

Проверяется не только подпись. Токен с верной подписью, но выданный другой областью или другому клиентскому приложению, — это чужой токен, и подпись у него правильная. Отдельно отклоняется токен личности: поле `typ` у него иное, тогда как подпись и область совпадают.

Проверяется	Отклоняется	Иначе возможно
подпись ключом области	подделанный токен	любой подписанный чем угодно токен считается действительным
срок действия	просроченный токен	отозванный доступ сохраняется бессрочно
издатель	токен чужой области	учётная запись из посторонней области получает доступ
получатель	токен для другой системы	токен, выданный для стороннего сервиса, принимается как свой
клиентское приложение	токен чужого приложения	приложение, зарегистрированное иначе, действует от имени системы

Таблица 6 — Состав проверки токена

После проверки токен дальше не передаётся: за периметром он уже проверен. Вместо него передаются три заголовка — идентификатор пользователя, имя и роли, — и те же сведения передаются далее в метаданных gRPC. Одноимённые заголовки, поступившие извне, затираются: иначе личность подделает любой, кому известны их наименования.

Идентификатор пользователя попадает в поле `actor` конверта события, поэтому в очереди видно, чьё действие привело к событию. У событий, порождённых обработкой другого события, поле остаётся пустым — человека там нет.

Проверка разнесена на два уровня, и это не дублирование: уровни отвечают на разные вопросы. Роль проверяет шлюз — вправе ли пользователь обращаться к такому запросу вообще. Принадлежность объекта проверяет владеющий им сервис — его ли это визит, его ли расписание.

Шлюзу принадлежность неизвестна и известна быть не должна.

Чтобы сервису было с чем сравнивать, шлюз выясняет у сервиса организации, какой сотрудник соответствует учётной записи, и передаёт его отдельным заголовком. Запрашивается это лишь для непривилегированных вызывающих: администратору и управляющему принадлежность безразлична, и обращаться к справочнику незачем. Ответ удерживается недолгое время — соответствие меняется редко, а требуется на каждом запросе.

Запрос	Владелец	Что проверяется
расписание сотрудника	организация	специалист видит только своё
карточка визита	записи	специалист видит только свои визиты
закрытие визита	записи	закрыть можно только свой визит
комиссия сотрудника	расчёты	специалист видит только свой заработок
загрузка сотрудника	аналитика	специалист видит только свою загрузку

Таблица 7 — Проверки принадлежности объекта

Проверка при закрытии визита предшествует изменению состояния. В противном случае событие о завершении успевае́т уйти в шину, и потребители начинают списывать материалы и выставл́ять счёт по чужому визиту. Отменить это уже нечем: событие доставлено. Отсюда и лишнее чтение перед изменением — выполняемое, впрочем, только для непривилегированного вызывающего.

Учётной записи может не соответствовать ни один сотрудник — обычное дело, не всякая учётная запись принадлежит работнику. В этом случае доступа к чужим объектам нет вовсе: отказ по умолчанию, иначе такая запись видела бы всё.

Отдельного упоминания заслуживает место проверки в сервисе организации. Она поставлена на границе внешнего интерфейса, а не внутри операции: ту же операцию вызывает сервис записей, когда вычисляет свободное время филиала, и чужое расписание там — совершенно законный запрос. Проверка внутри операции лишила бы специалиста возможности видеть свободное время коллег.

7.7 Наблюдаемость

Распределённая трассировка и сбор показателей подняты вместе с системой. В распределённой системе это не удобство, а единственный способ ответить на вопрос, где ушло время: журналы девяти процессов сами по себе в одну картину не складываются.

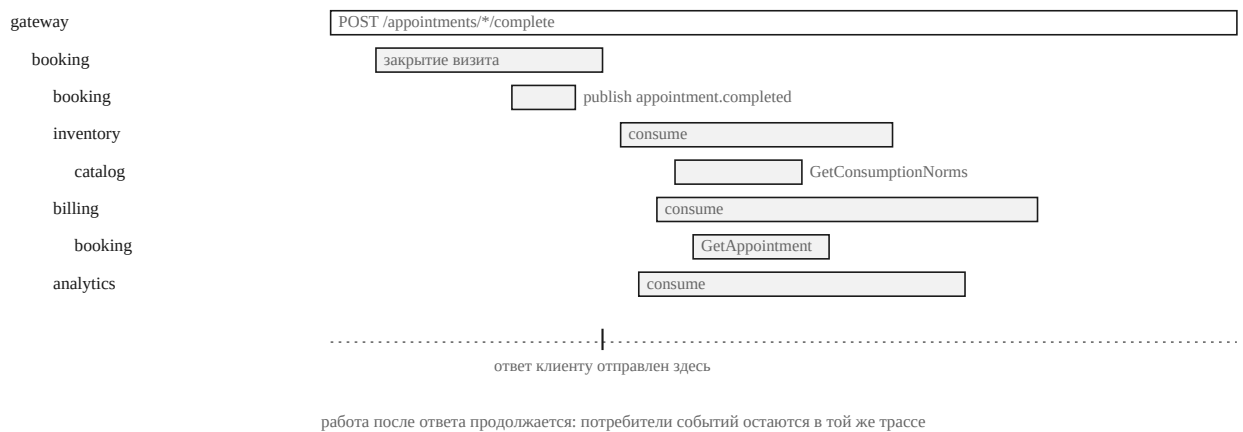


Рисунок 5 — Трасса одного запроса: участки работы сервисов на общей шкале времени

Существенно здесь то, что трасса не прерывается ни на границе языков, ни на границе очереди. Контекст передаётся в поле конверта события, и потребитель продолжает трассу издателя, а не начинает собственную. Механизм для этого существовал изначально: заголовок W3C передавался по всей системе задолго до подключения трассировки, и оставалось связать его с текущим участком.

Работа, выполняемая после отправки ответа, — списание материалов, выставление счёта, пересчёт витрин — остаётся в той же трассе. Без этого асинхронная часть системы оказалась бы вне наблюдения целиком.

Показатель	Что отражает
запросы по маршрутам	нагрузка по сервисам, маршрутам и кодам ответа
время обработки	распределение, из него вычисляется 95-й перцентиль
вызовы gRPC	обращения к соседям по методам и кодам
публикация событий	поток по ключам маршрутизации
обработка событий	исходы: обработано, пропущено, отказ, конверт не разобран
состояние базы	соединения, транзакции и размеры по каждой базе

Таблица 8 — Собираемые показатели

Меткой служит шаблон маршрута, а не фактический путь. В противном случае каждый идентификатор в адресе заводит собственный ряд значений, и объём хранимых показателей растёт вместе с числом записей в системе.

Отдельно учитываются два вида отказа потребителя. Первый — обработчик вернул ошибку. Второй — конверт события не удалось разобрать вовсе. Второй случай опаснее: сообщение уходит в очередь недоставленных молча, и заметить это без отдельного счётчика нечем. Оба вида проверены отправкой в шину неисполнимого и повреждённого события.

7.8 Непрерывная интеграция и доставка

Конвейер устроен так, что проверки предшествуют всему остальному: образ не публикуется, пока весь набор тестов не выполнен успешно. Работы выполняются параллельно по сервисам, поэтому время конвейера определяется самым медленным сервисом, а не их суммой.

Работа	Содержание
контракты	проверка описаний и соответствия сгенерированного кода
тесты Python	шесть сервисов: анализатор, модульные, интеграционные
тесты Go	три сервиса: форматирование, анализатор, обе группы тестов
публикация	образ службы склада в общедоступный реестр
развёртывание	справочник услуг бессерверным контейнером

Таблица 9 — Состав конвейера

Интеграционным тестам необходима база данных: она поднимается служебным контейнером, а базы с отдельными пользователями заводит тот же сценарий, что применяется при локальном запуске. Разделение тестов на группы, выполненное заранее, здесь окупилось: конвейер поднимает базу отдельным шагом, и переделывать сценарий не потребовалось.

Проверка соответствия сгенерированного кода описаниям контрактов заслуживает отдельного упоминания. Изменение описания без последующей генерации не проявляется у того, кто его внёс, — сборка ломается у следующего разработчика и в неожиданном месте. Конвейер выполняет генерацию и сравнивает результат с содержимым хранилища.

Выбор сервисов для двух способов доставки определяется их устройством. Служба склада удерживает соединение с брокером и подписку на очередь, поэтому остановка между вызовами для неё недопустима — сообщения накапливались бы в её отсутствие. Справочник услуг противоположен: отвечает на запросы, соединений не удерживает и в памяти процесса ничего не хранит, а потому переживает остановку без последствий.

Публикация включается наличием доступов. Учётные данные реестров хранятся в защищённых переменных хранилища, и при их отсутствии работы публикации пропускаются, а проверки выполняются полностью. Конвейер остаётся работоспособным и до настройки доступов, что позволяет проверять изменения с первого же дня.

8 Сквозные сценарии

8.1 Запись клиента

Синхронная часть работы системы: три сервиса, два вызова gRPC, одна проверка на уровне базы данных.

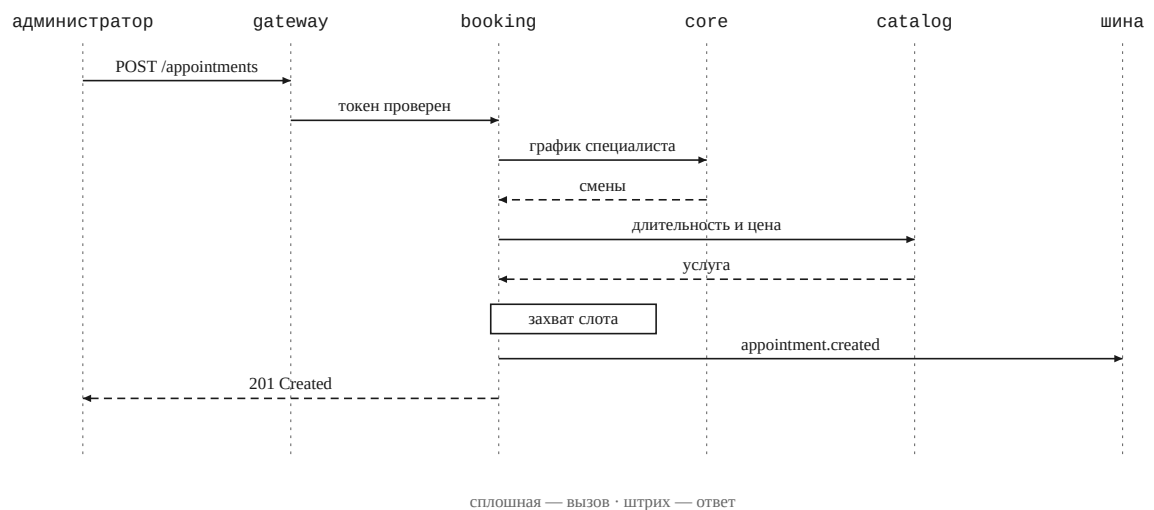


Рисунок 6 — Последовательность создания записи

Существенно, что захват слота выполняется ограничением базы данных, а не проверкой в коде: между чтением занятых интервалов и вставкой слот может быть занят другим запросом.

8.2 Завершение визита

Асинхронная часть: одно событие, три независимых потребителя.

1. Специалист закрывает запись, сервис `booking` публикует событие `appointment.completed`.
2. Сервис `inventory` запрашивает у каталога норматив расхода и списывает материалы одной транзакцией. При падении остатка ниже порога выпускает предупреждение в оба брокера.
3. Сервис `billing` запрашивает детали визита, выставляет счёт и рассчитывает комиссию специалиста.
4. Сервис `analytics` отмечает исход записи в витрине конверсии.
5. Сервис `notification` получает уведомление о счёте и о низком остатке, запрашивает контакты клиента и выполняет отправку.

Ни один из потребителей не осведомлён об остальных. Добавление четвёртого не требует изменений в издателе — достаточно нового связывания в топологии очередей.

9 Описание сервисов

Далее приводится спецификация каждого сервиса: область ответственности, принадлежность данных, программный интерфейс и решения, существенные для понимания его работы.

9.1 core-service

Ядро системы и единственный источник сведений об организации: какие существуют компании, филиалы, сотрудники и по какому графику они работают.

Язык	Python, FastAPI
База данных	core_db
Порты	8001 (HTTP), 9001 (gRPC)
Автотестов	35

Владеет. Компаниями, филиалами, сотрудниками, должностями, графиками работы.

Не владеет. Записями, клиентской базой, услугами. Ядро отвечает на вопрос «кто и где работает», а не «что происходит».

Программный интерфейс

Метод и путь	Назначение
POST /companies	зарегистрировать компанию
POST /companies/{id}/branches	открыть филиал
POST /branches/{id}/employees	принять сотрудника с графиком
GET /branches/{id}/employees	сотрудники филиала, фильтр по роли
GET /employees/{id}/schedule	график специалиста за период

Взаимодействие по gRPC

Предоставляет GetBranch и GetEmployeeSchedule сервису записей, ListEmployees — аналитике. Самый востребованный сервер gRPC в системе.

События

Публикует branch.opened и employee.hired. Событий не потребляет: ядро находится в начале цепочки, его состояние изменяют только пользователи.

Существенные решения

Смены специалиста не пересекаются, и это проверяется дважды: в агрегате при назначении графика, что даёт содержательное сообщение об ошибке, и ограничением исключения на уровне базы, что перехватывает запись в обход приложения. Дублирование намеренно: сервис записей принимает решение о занятости на основании этого графика, и пересекающиеся смены нарушили бы его логику. Смены, примыкающие встык, пересечением не считаются — интервал полуоткрытый.

Роли сотрудников совпадают с ролями в системе управления доступом. Справочник ядра и роли в токене должны согласовываться.

9.2 catalog-service

Справочник услуг: перечень оказываемых услуг, их стоимость, длительность и норматив расхода материалов на процедуру.

Язык	Python, FastAPI
База данных	catalog_db
Порты	8002 (HTTP), 9002 (gRPC)
Автотестов	13

Владеет. Услугами, ценами по филиалам, длительностью процедуры, справочником материалов и нормативами расхода.

Не владеет. Остатками на складе. Здесь норматив «сколько должно уйти», там факт «сколько есть и сколько израсходовано».

Программный интерфейс

Метод и путь	Назначение
POST /services	завести услугу вместе с нормативами
GET /services/{id}	карточка услуги
GET /branches/{id}/services	прайс-лист филиала
PUT /services/{id}/price	изменить цену

Взаимодействие по gRPC

GetService — длительность и цена при создании записи; GetConsumptionNorms — состав и количество материалов для списания после визита.

События

Событий не публикует и не потребляет. Не каждый сервис обязан участвовать в шине: это справочник, изменяемый только пользователями.

Существенные решения

Норматив расхода — содержательное ядро сервиса. Он связывает услугу со складом и делает списание автоматическим: администратор не отмечает вручную израсходованное количество, оно следует из услуги.

Отдельного интерфейса управления материалами нет. Материалы заводятся в ходе создания услуги и переиспользуются по наименованию, поскольку администратор мыслит услугой, а не складской номенклатурой.

Идентификатор филиала в запросе gRPC — проверка, а не фильтр. Услугу нельзя запросить от имени чужого филиала.

9.3 booking-service

Центральный сервис предметной области: запись клиентов к специалистам, расписание, расчёт свободных слотов. Единственный сервис, задействующий все четыре транспортных уровня.

Язык	Go, chi, pgx
База данных	booking_db
Порты	8003 (HTTP), 9003 (gRPC)
Автотестов	21

Владеет. Записями, занятостью слотов, историей визитов клиента.

Не владеет. Графиком специалиста и характеристиками услуги. Оба справочника запрашиваются синхронно в момент создания записи.

Программный интерфейс

Метод и путь	Назначение
GET /branches/{id}/slots	свободные слоты на период
POST /appointments	записать клиента
GET /appointments/{id}	детали записи
POST /appointments/{id}/complete	закрытие визита специалистом
POST /appointments/{id}/cancel	отмена с указанием причины

Взаимодействие по gRPC

Предоставляет `GetAppointment` расчётному сервису и `ListClientAppointments` — клиентскому. Сам обращается к ядру за графиком и к каталогу за услугой.

События

Публикует `appointment.created`, `appointment.cancelled`, `appointment.completed` в RabbitMQ и изменение занятости слота в NATS.

Существенные решения

Состязание за слот разрешается базой данных. Два администратора могут инициировать запись одновременно, и проверка в коде от этого не защищает. Ограничение исключения по специалисту и временному интервалу не позволит вставить пересекающуюся запись; проигравшая сторона получает код 409 с содержательным сообщением, а не код 500.

Отменённая запись освобождает слот, завершённая — нет. Условие входит в само ограничение. Ошибка в нём заблокировала бы время после каждой отмены, причём без внешних признаков, поэтому оно закрыто интеграционным тестом.

Сетка расписания получасовая. Смена, начинающаяся в 9:10, даёт первый слот в 9:30, иначе расписание распадается на непригодные промежутки. Запись должна целиком помещаться в одну смену.

Цена фиксируется в момент записи. Прайс-лист может измениться до визита, а счёт выставляется по цене, на которую клиент согласился.

9.4 inventory-service

Склад материалов филиала: списание по факту оказанной услуги, пополнение, контроль остатков. Первый потребитель событий в системе.

Язык	Go, chi, pgx
База данных	inventory_db
Порт	8004 (HTTP)
Автотестов	20

Владеет. Остатками материалов по филиалам, историей движений, минимальными порогами.

Не владеет. Справочником материалов и нормативами расхода — они принадлежат каталогу. Здесь учитываются только количества.

Программный интерфейс

Метод и путь	Назначение
GET /branches/{id}/stock	остатки филиала
GET /branches/{id}/stock/low	позиции ниже порога
POST /branches/{id}/stock/replenish	пополнить склад
PUT /branches/{id}/stock/{cid}/threshold	задать минимальный порог

Взаимодействие по gRPC

Обращается к каталогу за нормативом расхода. Собственного сервера gRPC не имеет.

События

Потребляет `appointment.completed`. Публикует `consumables.written_off` и `stock.low`, а также предупреждение в NATS.

Существенные решения

Списание и пополнение реализованы разными запросами. Одной операцией вставки с разрешением конфликта они не выполняются: ограничение неотрицательности проверяется на предлагаемой строке до разрешения конфликта, а конструкция разрешения конфликта перехватывает только нарушения уникальности. При списании сервер обнаружит отрицательное количество и завершит операцию ошибкой, не дойдя до обновления. Поэтому пополнение создаёт позицию, а списание лишь уменьшает существующую, что соответствует и смыслу операции.

Все движения выполняются одной транзакцией. Частично списанный набор материалов хуже, чем не списанный вовсе.

Нехватка материала — тупиковая ситуация, а не повод для повтора. Событие направляется в очередь недоставленных и требует вмешательства: пополнить склад и повторить обработку.

Один факт передаётся в оба брокера по-разному. Низкий остаток уходит в RabbitMQ как доменное событие для отчётности и уведомления, в NATS — как мгновенное предупреждение на экран.

9.5 client-service

Клиентская база: карточка клиента, контактные данные, программа лояльности. Первый сервис, одновременно являющийся и сервером, и клиентом gRPC.

Язык	Python, FastAPI
База данных	client_db
Порты	8005 (HTTP), 9005 (gRPC)
Автотестов	14

Владеет. Персональными данными клиента, контактами, предпочитаемым каналом связи, балансом и историей начисления баллов.

Не владеет. Историей записей — она принадлежит сервису записей. Карточка получает её по gRPC, а не хранит копию.

Программный интерфейс

Метод и путь	Назначение
POST /clients	завести клиента
GET /clients?phone=	поиск по телефону
GET /clients/{id}	карточка с историей записей
GET /clients/{id}/loyalty	баланс и история начислений

Взаимодействие по gRPC

Предоставляет GetClientContacts сервису уведомлений и AddLoyaltyPoints — расчётному. Сам запрашивает историю записей у сервиса записей.

События

Публикует client.registered. Событий не потребляет.

Существенные решения

Баллы начисляет не этот сервис. Сумму оплаты знает расчётный сервис, поэтому он и инициирует начисление, тогда как хранит баллы клиентский. Разделение «кто знает» и «кто хранит» проходит через всю систему.

Начисление идиempotentно по номеру счёта. Расчётный сервис может повторить вызов при повторной попытке, но повторное начисление вернёт нулевое приращение и текущий баланс. Состязание двух одновременных попыток разрешается уникальным индексом.

Единственное место хранения персональных данных. Это заметно упрощает разграничение доступа: защищать требуется один сервис.

9.6 billing-service

Расчёты за визит: выставление счёта по завершённой записи, отметка оплаты, расчёт комиссии специалиста.

Язык	Python, FastAPI
База данных	billing_db
Порт	8006 (HTTP)
Автотестов	15

Владеет. Счетами, платежами, расчётом комиссии специалиста.

Не владеет. Ценой услуги и балансом баллов клиента. Первое запрашивается у сервиса записей, второе изменяется через клиентский сервис.

Программный интерфейс

Метод и путь	Назначение
GET /invoices/{id}	счёт
POST /invoices/{id}/pay	отметить оплату
GET /branches/{id}/invoices	счета филиала, фильтр по статусу
GET /employees/{id}/commission	комиссия специалиста за период

Взаимодействие по gRPC

Обращается к сервису записей за деталями визита и к клиентскому сервису за начислением баллов. Собственного сервера gRPC не имеет.

События

Потребляет `appointment.completed`. Публикует `invoice.issued` и `invoice.paid`.

Существенные решения

Комиссия начисляется только по оплаченным счетам. Выставленный, но не оплаченный счёт комиссии не порождает — иначе специалисту начислялось бы вознаграждение за неполученные деньги.

Идемпотентность обеспечена на двух уровнях. Таблица обработанных событий не допускает повторной обработки, а уникальность идентификатора записи в счетах служит страховкой на уровне схемы. Один визит — один счёт.

Отметка оплаты — синхронная операция с побочным эффектом у соседа. Она инициирует начисление баллов и только затем публикует событие оплаты. Порядок существен: событие означает, что все действия уже выполнены.

9.7 notification-service

Взаимодействие с внешними каналами связи: напоминания клиентам, предупреждения администраторам филиала. Единственный сервис, слушающий оба брокера, и единственный без базы данных.

Язык	Go, chi
База данных	не используется
Порт	8007 (HTTP)
Автотестов	11

Владеет. Шаблонами сообщений и логикой выбора канала связи.

Не владеет. Контактными данными клиента — они запрашиваются в момент отправки. Собственного состояния сервис не имеет вовсе.

Программный интерфейс

Метод и путь	Назначение
POST /notifications	отправить произвольное уведомление
GET /notifications/{id}	статус отправки
GET /templates	перечень шаблонов

Взаимодействие по gRPC

Запрашивает контакты клиента у клиентского сервиса.

События

Потребляет `appointment.created`, `appointment.cancelled`, `invoice.issued`, `stock.low` из RabbitMQ и предупреждения из NATS.

Существенные решения

Выбор канала учитывает предпочтение клиента, но допускает замену. Незаполненный адрес предпочитаемого канала не является основанием отказать от отправки. Администратору филиала личные контакты не требуются, его сообщения выводятся на экран у стойки регистрации.

Два вида отказов трактуются по-разному, и это главное решение сервиса. Недоступность почтового сервера помечает отправку неудачной, но ошибка наружу не поднимается: поток событий важнее одного сообщения. Недоступность клиентского сервиса поднимает ошибку, и событие направляется в очередь недоставленных: без контактных данных отправка невозможна, а повтор осмыслен. Отсутствие различия приводит либо к переполнению очереди из-за отказа почтового сервера, либо к безвозвратной потере уведомлений.

Отсутствие базы данных обоснованно. Собственного состояния у сервиса нет, журнал отправок хранится в кольцевом буфере на 500 записей и теряется при перезапуске. Отсюда и наименьший образ в системе — 28 МБ.

9.8 analytics-service

Отчётность по сети: выручка филиалов, загрузка специалистов, расход материалов, конверсия записей. Единственный подписчик на весь поток событий.

Язык	Python, FastAPI
База данных	analytics_db
Порт	8008 (HTTP)
Автотестов	16

Владеет. Витринами данных. Собственных бизнес-фактов не создаёт: всё состояние производное.

Не владеет. Ничем первичным. Любой показатель выводится из событий; в чужие базы данных аналитика не обращается.

Программный интерфейс

Метод и путь	Назначение
GET /branches/{id}/revenue	выручка, число счетов, средний чек
GET /employees/{id}/workload	записи, завершённые, занятое время, комиссия
GET /branches/{id}/funnel	конверсия записей
GET /reports/consumables	расход материалов по позициям

Взаимодействие по gRPC

Обращается к ядру за наименованиями филиалов и сотрудников: витрины хранят только идентификаторы.

События

Потребляет весь поток событий системы.

Существенные решения

Время факта берётся из события, а не из момента записи. Первоначально использовалась отметка времени по умолчанию, и дефект был обнаружен тестом: при отставании сервиса и последующей обработке накопленной очереди все факты пришлось бы на дату обработки, исказив отчётность за период. Показатели при этом сохраняли бы правдоподобие.

Одна строка на сущность, а не на событие. Витрина записей хранит жизненный цикл, поэтому конверсия вычисляется группировкой по состоянию, а не вычитанием счётчиков.

Событие исхода может опередить событие создания. Очереди у сервисов разные, порядок между ними не гарантирован. При отсутствии факта записи завершение не выполняет никаких действий.

Выручка учитывается по оплаченным счетам, занятое время — по завершённым записям, незавершённые записи не влияют на конверсию. Каждое из трёх правил изменяет значение в отчёте.

9.9 gateway

Единственная точка входа: проверка токена и маршрутизация. Инфраструктурный компонент, в число восьми микросервисов не входит, собственной предметной области не имеет.

Язык	Python, FastAPI
База данных	не используется
Порты	8000 — единственный, открытый вовне
Автотестов	50
Относится к работе	ПР9

Владеет. Проверкой прав доступа и таблицей маршрутов.

Не владеет. Никакими данными предметной области.

Программный интерфейс

Метод и путь	Назначение
POST /auth/token	Выдача токена по имени и паролю
GET /routes	Таблица маршрутов с требуемыми ролями
* /...	Всё остальное передаётся в сервисы

Существенные решения

Обоснование отдельного компонента. Ядром системы задуман core, однако проксирование через него чужих запросов нежелательно: обновление ядра остановило бы всю систему, а в трассировке каждый запрос проходил бы через ядро дважды. Это и есть распределённый монолит, за который микросервисную архитектуру критикуют.

Разделение ролей. Шлюз остаётся тонким, без бизнес-логики. Сервис core сохраняет положение центра в более существенном смысле — как единственный источник сведений об организации, к которому синхронно обращаются остальные.

Передача личности пользователя. Внутри системы сервисы взаимодействуют по gRPC, и без передачи сведений о пользователе внутренняя сеть окажется полностью доверенной: получивший доступ внутрь периметра сможет действовать от чужого имени.

Два адреса системы управления доступом. В поле издателя токена проставляется тот адрес, по которому обратился браузер, тогда как ключи проверки подписи шлюз получает изнутри сети. Адреса различны, и сверять поле издателя следует с внешним. В противном случае токен, полученный изнутри сети, оказывается выданным иным издателем, нежели токен из браузера, и проверка их не сводит.

Таблица маршрутов как модель прав. Соответствие «путь — сервис — требуемая роль» описано в одном месте и доступно для чтения по отдельному адресу. Права системы читаются целиком, а не собираются по восьми хранилищам исходного кода.

10 Незавершённые работы

Все восемь сервисов реализованы, покрыты автоматическими тестами и работают совместно. Доступ извне закрыт шлюзом, система наблюдаема, конвейер выполняет проверки на каждом изменении. Сквозные слои, выполняемые однократно для всей системы, завершены; оставшееся сводится к настройке внешних учётных данных.

Что осталось	Почему не выполнено сейчас
Публикация образов	работы конвейера написаны и проверены, включаются наличием учётных данных внешних реестров

Таблица 10 — Оставшиеся работы

Оставшийся пункт не требует изменений в коде системы: работы конвейера написаны и проверены, публикация включается появлением учётных данных внешних реестров. Публичный интерфейс описан спецификацией OpenAPI, полнота которой проверяется конвейером: маршрут, добавленный в шлюз мимо описания, роняет сборку.